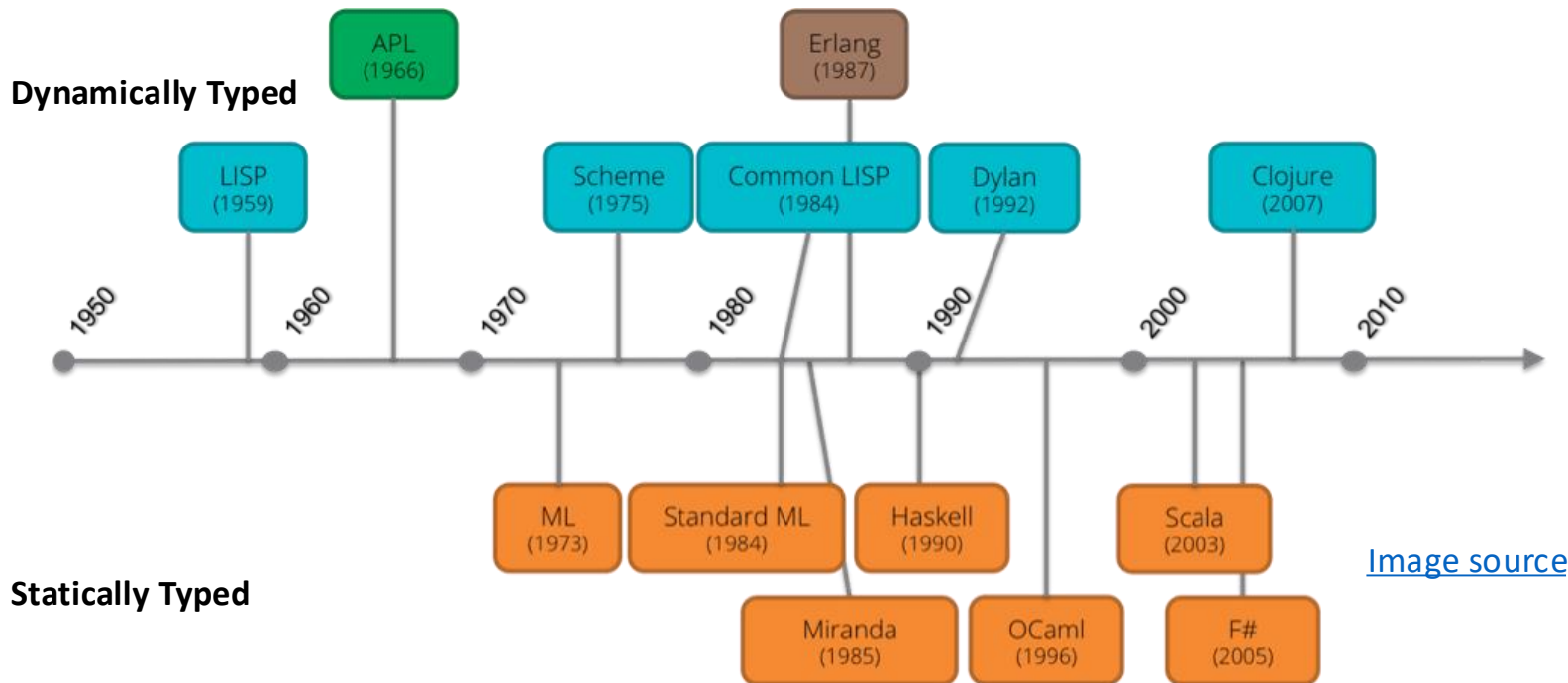


CS571 Programming Languages

Spring 2026

9. Functional Programming II: Scheme

Functional Languages



Running Scheme

- Racket: a Scheme-based language
- <https://racket-lang.org/>
- DrRacket: IDE for Racket



- Online:
<https://onecompiler.com/racket>

A screenshot of the DrRacket IDE window. The title bar reads "Untitled - DrRacket". The menu bar includes "Untitled", "(define ...)", a save icon, a search icon, "Debug", a play icon, "Macro Stepper", a run icon, and a stop icon. The editor area shows two lines of code: line 1 is "#lang racket" and line 2 is "(display \"Welcome to Functional Programming\")". The output area below shows the execution results: "Welcome to DrRacket, version 8.11.1 [cs].", "Language: racket, with debugging; memory limit: 128 MB.", and "Welcome to Functional Programming". The status bar at the bottom indicates "Determine language from source", "1:41", and "524.04 MB".

```
1 #lang racket
2 (display "Welcome to Functional Programming")
```

Welcome to DrRacket, version 8.11.1 [cs].
Language: racket, with debugging; memory limit: 128 MB.
Welcome to Functional Programming
>

Functional Programming in Scheme

- We focus on the non-imperative subset of Scheme (no assignments, loops)
 - Definitions: (`define` ...)
 - Functions: (`lambda` (...) ...)
 - Conditional: (`if`) (`cond` [... ..] [... ..] [`else` ...])
 - List: (`cons`), (`car` ...), (`cdr` ...)
 - Local Bindings (`let` ...) (`let*` ...) (`letrec` ...)

Scheme

Syntax:

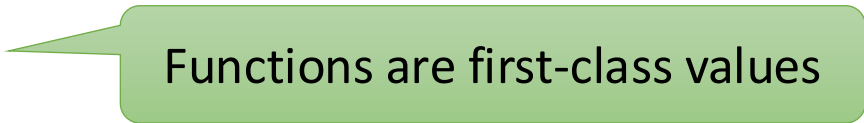
- Atoms: e.g., 3, #t, #f, “abc”
- Lists: (3 #t “abc”)
- Functions: (sqrt 2), (+ 1 2 4)
- Comments: (+ 1 2) ; evaluate to 3



Prefix form

Parenthesis in Scheme

- In Scheme, () is used for lists, and both code and data are *lists*
 - (+ 2 3)
 - (2 3 4)
- Lists are evaluated as function calls
 - (+ 2 3) ; evaluates to 5
 - (2 3 4) ; Exception: 2 is not a procedure
 - ((+ 2 3)) ; Exception: 5 is not a procedure



Functions are first-class values

Parenthesis in Scheme

- Lists w/o evaluation: use apostrophe (')
 - `'(2 3 4)` ; returns `(2 3 4)`
- Read-Eval-Print Loop (REPL)
 - `(sqrt 2)` is the same as `(eval '(sqrt 2))`

Expressions

- Arithmetic:

```
(* 1 2 3 4)  
(+ (* 1 2) (* 3 4))  
(/ 1 2)
```
- Relational:

```
(< (* 1 2) (+ 1 1))  
(and (> 2 4) #t)
```
- Conditional:

```
(if (< 2 3) 1 (+ 2 3))  
(* 2 (if (< 3 5) 3 "abc"))
```
- Type test:

```
(number? 2)          (number? #f)  
(string? "a")        (boolean? 1)  
(list? '(1 2 3))
```


Environment

- Environment binds identifiers to values
 - $+$, $*$, $-$, $/$ are identifier bound to values
 - Functions are values in Scheme
- Built-in identifiers in initial environment
 - $+$, sqrt , positive? , max , ...
- Add bindings to the environment
 - Use key word: **define**

Definition

- Keyword `define` introduces global identifiers

```
(define pi 3.14159)
(define radius 2)
(* pi (* radius radius))
(define circumference (* 2 pi radius))
circumference
```

Local Definition

Let expr: (**let** ((x 3)
 (y (sqrt 7)))
 (+ x y))

General form: (**let** ((x1 expr1)
 (x2 expr2)
 ...
 (xn exprn))
 body-expr)

x1, x2, ..., xn can be used in body-expr

Local Definition

Let* expr: (**let*** ((x 3)
 (y x))
 (+ x y))

General form: (**let*** ((x1 expr1)
 (x2 expr2)
 ...
 (xn exprn))
 body-expr)

x1 can be used in expr2, expr3, ..., body-expr

x2 can be used in expr3, ..., body-expr

Anonymous Function

Anonymous functions are introduced by `lambda`

```
(lambda (x y z) expr)
```

parameter

function body

Example

```
((lambda (x) x) 1)  
((lambda (x y z) (+ x y z)) 1 2 3)  
((lambda (f) (f 2)) (lambda (x) (* 2 x)))
```

Named Function

- In Scheme, function is a first-class value, so similarly, we can define named functions

```
(define f (lambda (x) (+ x 1)))  
(f 1)
```

```
(let ((f (lambda (x) (+ x 1)))  
      (g (lambda (y) (* y 2))))  
  (+ (f 1) (g 2)))
```

Named Function

- In Scheme, named functions can be introduced in a more concise way:

```
(define (f x) (+ x 1))
```



func. name

parameter

func. body

- Is the same as

```
(define f (lambda (x) (+ x 1)))
```



Named Function

```
(define (sqr x) (* x x))  
(sqr 5)  
(define (area x y) (* x y))  
(area 5 10)
```

Common use:

```
(define (f x y z)  
  (let ((x1 exp)  
        (x2 exp))  
    f's body))
```




Multiple Branches with Cond

```
(define (abs x)
  (cond [(< x 0) (- x)]
        [else x]))
```

```
(define (test x)
  (cond [(number? x) "num"]
        [(string? x) "str"]
        [(list? x) "list"]
        [else "other"])))
```

Scheme: Key Points

- Evaluation of expressions
 - $(e_0 \ e_1 \ e_2 \ e_3)$
- Key words
 - `define`, `if`, `cond`, `'`
- No static type system

```
(define (f x) (+ x "abc"))  
(* 2 (if (< 3 5) 3 "abc"))
```

Scheme: Key Points

- No assignments, no iterations (loops)
- All variables are immutable (mathematical symbols)

Recursion and Induction

- Recursion
 - Computationally: a procedure/function that calls itself
 - Semantically: defining a computation/value inductively

Recursion and Induction

- Induction

- Natural numbers

- 1) $0 \in \mathbb{N}$

- 2) If $n \in \mathbb{N}$, then $n + 1 \in \mathbb{N}$

- Inductive proof that property P holds for all $n \in \mathbb{N}$

- 1) P holds for 0

- 2) If $n \in \mathbb{N}$ and P holds for n , then P holds for $n + 1$

Factorial

- $n! = n * (n - 1) * \dots * 1$
- Inductive definition:
 - $0! = 1$
 - For $n > 0$, $n! = n * (n-1)!$

```
(define (factorial n)
  (if (= n 0) 1
      (* n (factorial (- n 1)))))
```

Fibonacci Number

- 1, 1, 2, 3, 5, 8, 13, 21,...
- Inductive definition:
 - $\text{fib}(1) = 1, \text{fib}(2) = 1$
 - $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ for $n > 2$

```
(define (fib n)
  (cond [(= n 1) 1]
        [(= n 2) 1]
        [else (+ (fib (- n 1)) (fib (- n 2)))]))
```

Lists

- A list is a sequence of expressions in parentheses

`(1 2 3)` `(sqrt x)` `(x 1 “abc”)`

- Programs are just lists interpreted as code

Lists

- A list is either
 - Empty: ' ()
 - Non-empty: it has a *head* and a *tail*, where:
 - The *head* can have any type
 - The *tail* is itself a list
 - Inductive definition!

List Construction

- `null/empty/'()` is a list
- If `x` is an expression, `xs` is a list, then

`(cons x xs)` is a list

head

tail

- “`cons`” is a built-in function for constructing lists
`(1 2 3 4)` is equivalent to
`(cons 1 (cons 2 (cons 3 (cons 4 '()))))`

List Destruction

- `car` and `cdr` are the destructors for lists
- If `xs` is a non-empty list, then
 - `(car xs)` is the `head`
 - `(cdr xs)` is the `tail`
- Algebraically:
 - $(\text{car } (\text{cons } x \text{ xs})) = x$
 - $(\text{cdr } (\text{cons } x \text{ xs})) = \text{xs}$

List Destruction

- $(\text{car } '(1\ 2\ 3)) = 1$, which is the same as
 $(\text{car } (\text{cons } 1\ '(2\ 3))) = 1$
- $(\text{cdr } '(1\ 2\ 3)) = (2\ 3)$, which is the same as
 $(\text{cdr } (\text{cons } 1\ '(2\ 3))) = (2\ 3)$
- $\text{caar, cadr, cdar, cddr, caaar} \dots$

Encoding: List

- $\text{NULL} \triangleq \text{FALSE} \triangleq \lambda x y. y$
- $\text{CONS} \triangleq \text{PAIR} \triangleq \lambda x y. \lambda f. f x y$
- $\text{CAR} \triangleq \text{LEFT} \triangleq \lambda p. p \lambda x y. x$
- $\text{CDR} \triangleq \text{RIGHT} \triangleq \lambda p. p \lambda x y. y$
- $\text{ISNULL} \triangleq \lambda l. l (\lambda h t. \lambda d. \text{FALSE}) \text{TRUE}$

List Functions

- **cons** adds an element to a list
- **car** returns the head of a non-empty list
- **cdr** returns the tail of a non-empty list
- **append** concatenates two lists

```
> (cons 1 '(2 3 4))  
'(1 2 3 4)  
> (car '(1 2 3 4))  
1  
> (cdr '(1 2 3 4))  
'(2 3 4)  
> (append '(1 2) '(3 4))  
'(1 2 3 4)
```

Additional List Functions

- `list` creates a list from its arguments
- `null?` checks if a list is empty
- `list?` checks if something is a list

```
> (list 1 2 3)
'(1 2 3)
> (null? '())
#t
> (list? '())
#t
```

Equality Test

- `=`: check if two numbers are equal
- `equal?`: return `#t` if two expressions have the same **structure and content**
- `eq?`: check if two parameters represent the same object in memory

```
> (= 1 1.0)
#t
> (equal? 1 1.0)
#f
> (equal? '(1) (cons 1 null))
#t
> (equal? '(1) '(1 (2 3)))
#f
> (define pi 3.14)
> (eq? pi 3.14)
#t
```




Working with Lists

- Any thing we want to do with a list, we need to deal with:
 - What do we do with an empty list? (null case)
 - What do we do with a non-empty list?
 - How do we use head/tail to solve the problem?

List Example

- Define `lstSum`, which returns the sum of a num list
- Inductive definition
 - `'(): 0`
 - `(cons head tail): head + lstSum(tail)`

```
(define (lstSum lst)
  (if (null? lst) 0
      (+ (car lst) (lstSum (cdr lst)))))
```



Recursion over Lists

- General format

```
(define (lstFunc lst)
  (if (null? lst) (base-case)
      (recursive-case using (car lst) (cdr lst))))
```

More List Functions

- Define a function `subs`, which takes a `b` `lst`, and replaces `a` with `b` in `lst`

```
(define (subs a b lst)
  (if (null? lst) '()
      (if (equal? (car lst) a)
          (cons b (subs a b (cdr lst)))
          (cons (car lst) (subs a b (cdr lst))))))
```

Higher Order Functions

- In Scheme, function is a **first-class value**
 - Functions can go wherever expressions go
- Higher-Order Functions
 - Languages where it allows a function to be:
 - The return value of a function (first order)
 - An argument to another function (second order)

Function as Argument

- Repeat any function twice?

```
(define (double x) (* x 2))  
(define (quad x) (double (double x)))  
(define (square x) (* x x))  
(define (fourth x) (square (square x)))
```

- More reusable code

```
(define (twice f x) (f (f x)))  
(twice double 2)  
(twice square 2)
```

Examples w/o High-Order Functions

- Find all numbers > 5 in a list

```
newlst = empty;  
foreach (elem in lst):  
    if (elem > 5):  
        add elem to newlst;
```

- Find all strings in a list

```
newlst = empty;  
foreach (elem in lst):  
    if (elem is string):  
        add elem to newlst;
```



Filter

`(filter pred lst)`: returns a new list with elements from `lst` for which `pred` returns `#t`



Examples with High-Order Functions

```
> (filter (lambda (x) (> x 5)) '(1 4 7 10))  
'(7 10)  
> (filter string? '(1 "a" 2 "b" 3 "c"))  
'("a" "b" "c")
```

Examples w/o High-Order Functions

- Multiply each element in a list by 2

```
newlst = empty;  
foreach (elem in lst):  
    add elem*2 to newlst;
```

- Square each element in a list

```
newlst = empty;  
foreach (elem in lst):  
    add elem*elem to newlst;
```



Map

`(map func lst)`: applies function `func` to each element of list `lst`

Examples with High-Order Functions

```
> (map double '(1 2 3))  
'(2 4 6)  
> (map square '(1 2 3))  
'(1 4 9)
```

Map

- In previous slides we had:

```
(define (subs a b lst)
  (if (null? lst) '()
      (if (equal? (car lst) a)
          (cons b (subs a b (cdr lst)))
          (cons (car lst) (subs a b (cdr lst))))))
```

- Using map, we have

```
(define (subs a b lst)
  (map (lambda (x) (if (equal? a x) b x)) lst))
```

Map on Multiple Lists

`map f l1 l2 l3`: applies function `f` to elements at the same position of list `l1, l2, l3` (with same length)

```
> (map + '(1 2 3) '(4 5 6) '(7 8 9))  
'(12 15 18)
```

```
> (map (lambda (x y) (- x y)) '(1 2 3) '(4 5 6))  
'(-3 -3 -3)
```

Examples w/o High-Order Functions

- Compute the sum of a list

```
init = 0;
foreach (elem in lst):
    init += elem;
```

- Compute the product of a list

```
init = 1;
foreach (elem in lst):
    init *= elem;
```

Foldr (Fold Right)

`(foldr f z ' (x1 x2 ... xn))`: returns
`(f x1 (f x2 ... (f xn z)))`

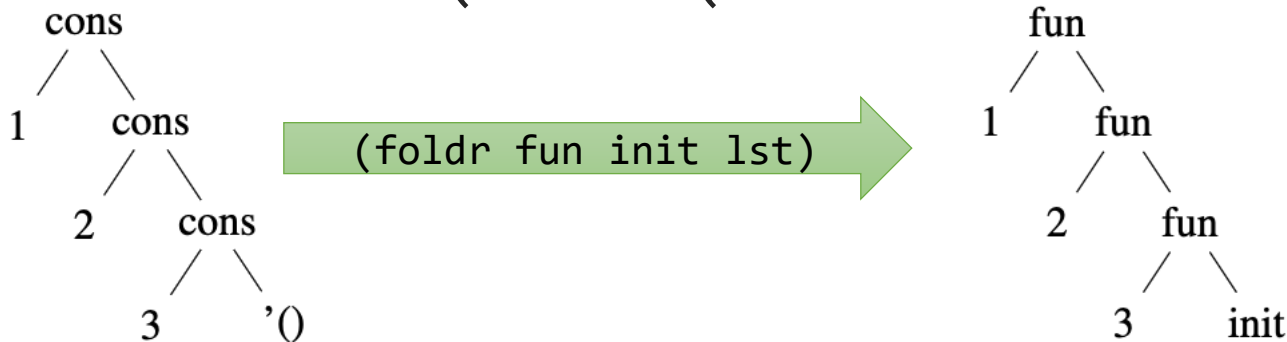
Foldr

▪ '(1 2 3):

(cons 1 (cons 2 (cons 3 null)))

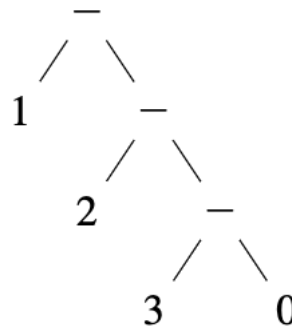
▪ (foldr fun init '(1 2 3)):

(fun 1 (fun 2 (fun 3 init)))



Foldr

```
> (foldr - 0 '(1 2 3))  
2  
> (foldr cons '() '(1 2 3))  
'(1 2 3)
```



Foldl (Fold Left)

$(\text{foldl } f \ z \ ' (x_1 \ x_2 \ \dots \ x_n))$: returns
 $(f \ x_n \ \dots (f \ x_2 \ (f \ x_1 \ z)))$

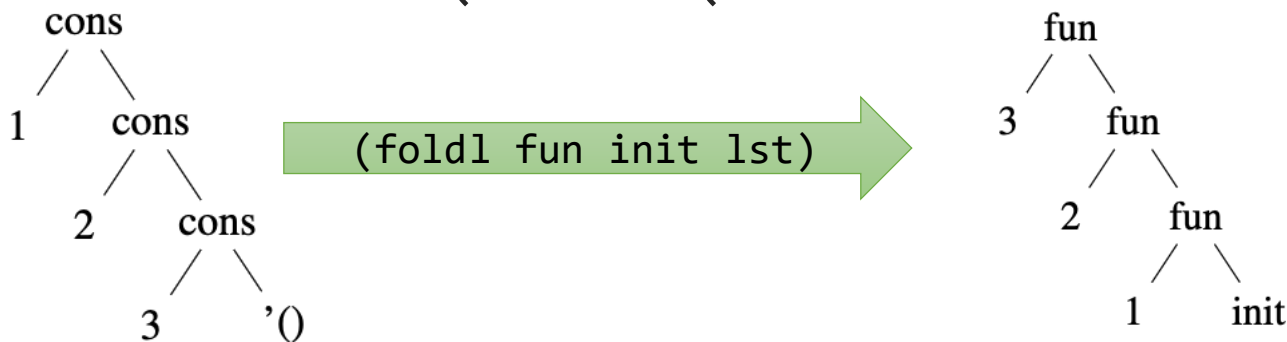
Foldl

▪ '(1 2 3):

(cons 1 (cons 2 (cons 3 null)))

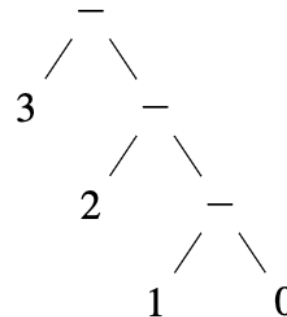
▪ (foldl fun init '(1 2 3)):

(fun 3 (fun 2 (fun 1 init)))



Foldl

```
> (foldl - 0 '(1 2 3))  
2  
> (foldl cons '() '(1 2 3))  
'(3 2 1)
```



Example

- In previous lecture, we had:

```
(define (lstSum lst)
  (if (null? lst) 0
      (+ (car lst) (lstSum (cdr lst)))))
```

- Using fold, we have:

```
(define (lstSum lst)
  (foldl + 0 lst))
```

Example

`(foldl f z '(x1 x2 ... xn)):`
returns `(f xn ... (f x2 (f x1 z)))`

```
(foldl (lambda (x a) (+ a (* x x)))  
0 '(1 2 3))
```

Function as Return Value

```
> (define (addN n) (lambda (m) (+ m n)))
```

```
> ((addN 10) 20)
```

```
30
```

```
> (twice (addN 10) 20)
```

```
40
```

```
> (+ 10 20)
```

```
30
```

```
> (twice (+ 10) 20)
```

```
❗ ❗ application: not a procedure;  
expected a procedure that can be applied to arguments  
given: 10
```


Multi-Argument Functions

- A multi-argument function can only be used when all parameters are ready!

```
> (define (add m n) (+ m n))
```

```
> (add 1 2)
```

```
3
```

```
> (add 1)
```

❗ ❗ *add: arity mismatch;*

*the expected number of arguments does
not match the given number*

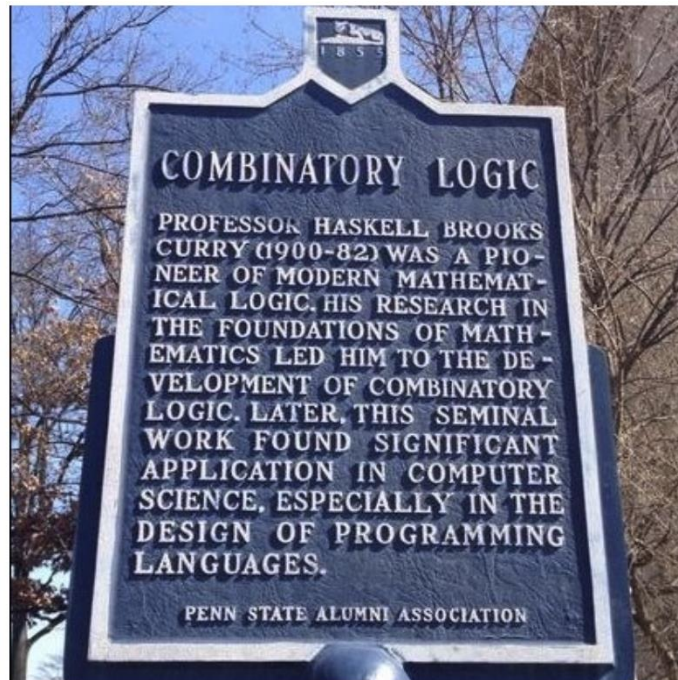
expected: 2

given: 1

Currying



Haskell B. Curry
Penn State 1929-1966



Outside of McAllister Building

Currying

- Currying: every function is treated as taking at most *one* parameter

```
// Uncurried version  
// add: int,int->int
```

```
(define (add m n)  
  (+ m n))
```

```
(add 2 3) // OK
```

```
(add 2)   // Error
```

```
// Curried version  
// addc: int->(int->int)
```

```
(define (addc m)  
  (lambda (n) (+ m n)))
```

```
((add 2) 3) // OK
```

```
(add 2)     // OK
```

Curried form allows partial evaluation

Currying

In terms of lambda calculus, the curried function of $\lambda x_1 x_2 \dots x_n. e$ is $\lambda x_1. (\lambda x_2. (\dots (\lambda x_n. e)))$

```
(define (curry2 f)
  (lambda (x)
    (lambda (y)
      (f x y)))))
```

```
(define (curry3 f)
  (lambda (x)
    (lambda (y)
      (lambda (z)
        (f x y z))))))
```

Partial Evaluation

- A function is evaluated with one or more of the leftmost actual parameters
 - `( (curry2 add) 2 )` is a partial evaluation of `add`
- We can think it as a temporary result, in the form of a function

Uncurrying

In terms of lambda calculus, the uncurried function of $\lambda x_1. (\lambda x_2. (\dots (\lambda x_n. e)))$ is $\lambda x_1 x_2 \dots x_n. e$

```
(define (uncurry2 f)
  (lambda (x y)
    ((f x) y)))
```

```
(define (uncurry3 f)
  (lambda (x y z)
    (((f x) y) z)))
```